ENCRYPTED DIRECT MESSAGING SYSTEM

Swing Desktop GUI, Software Design, Testing and Graduate Attribute Reflection

SDN260S - Software Design II

Group 27

26 October 2026 | Java 17 | Swing Desktop GUI Trial

DRAFT FOR GROUP VERIFICATION - Swing GUI and 23 passing tests

Prepared from the group's supplied Java console prototype and UML image. Review all names, authorship, technical claims and reflection before submission.

Original UML Class Diagram

This is the group's supplied five-class model. The implementation adds explicit authentication, network and logging services and refines several field types.

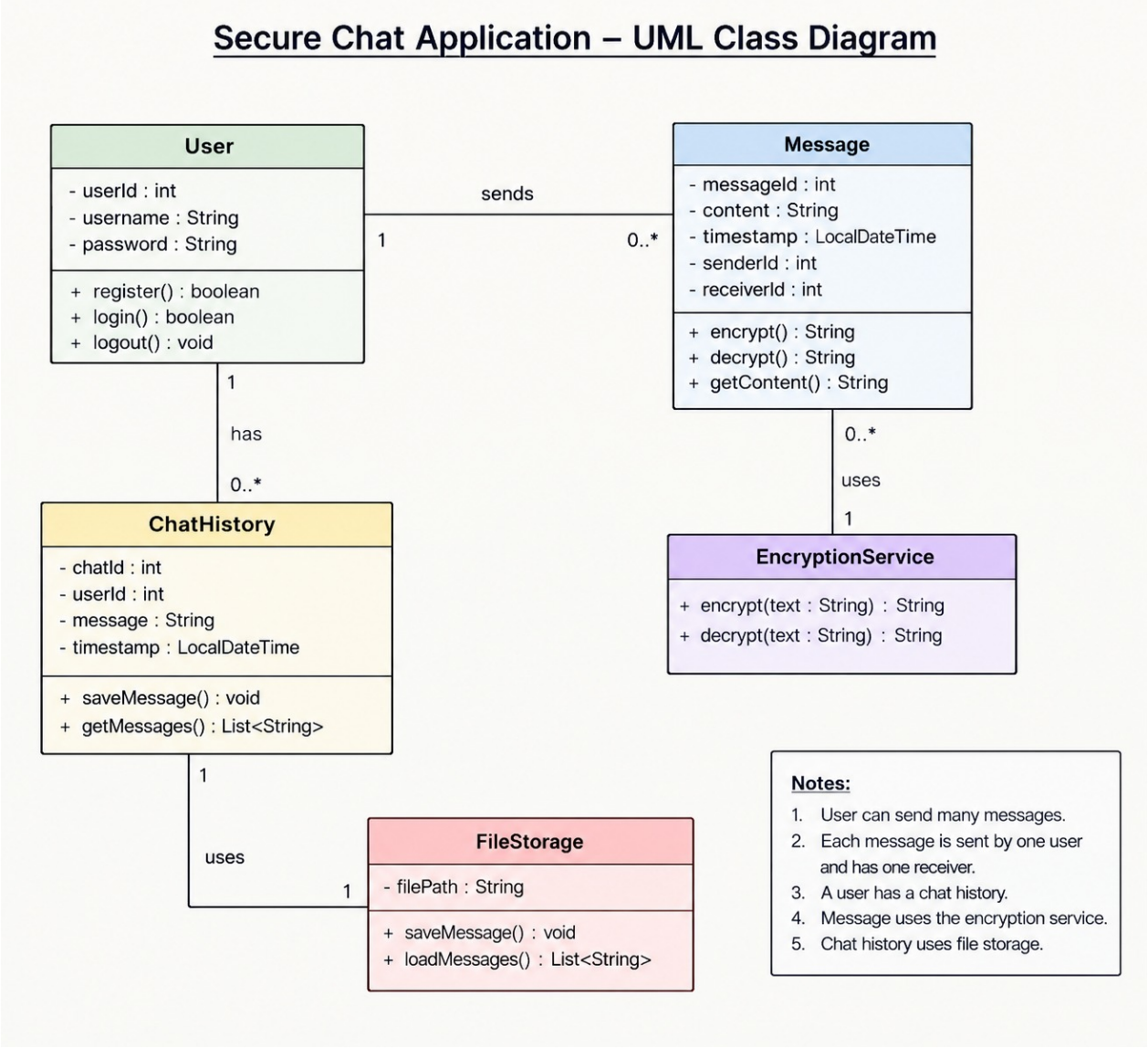


Figure 1. Original supplied Secure Chat Application UML.

Implementation mapping

User / AuthManager handles accounts; Message stores the UUID, participants, time and content; ChatHistory uses FileStorage for encrypted records; EncryptionService applies AES-GCM; NetworkManager handles sockets; AuditLogger keeps operational events. EdmsGui is the new Swing desktop view/controller. The exact updated diagram is provided in UML_UPDATED.md with the source archive.

New Swing Desktop Chat Interface

The new UI adds account registration/login, server/client connection setup, encrypted chat bubbles, a history viewer and disconnect/logout controls. This image is rendered from the actual Swing ChatPanel component with example messages, not captured from a live chat session.

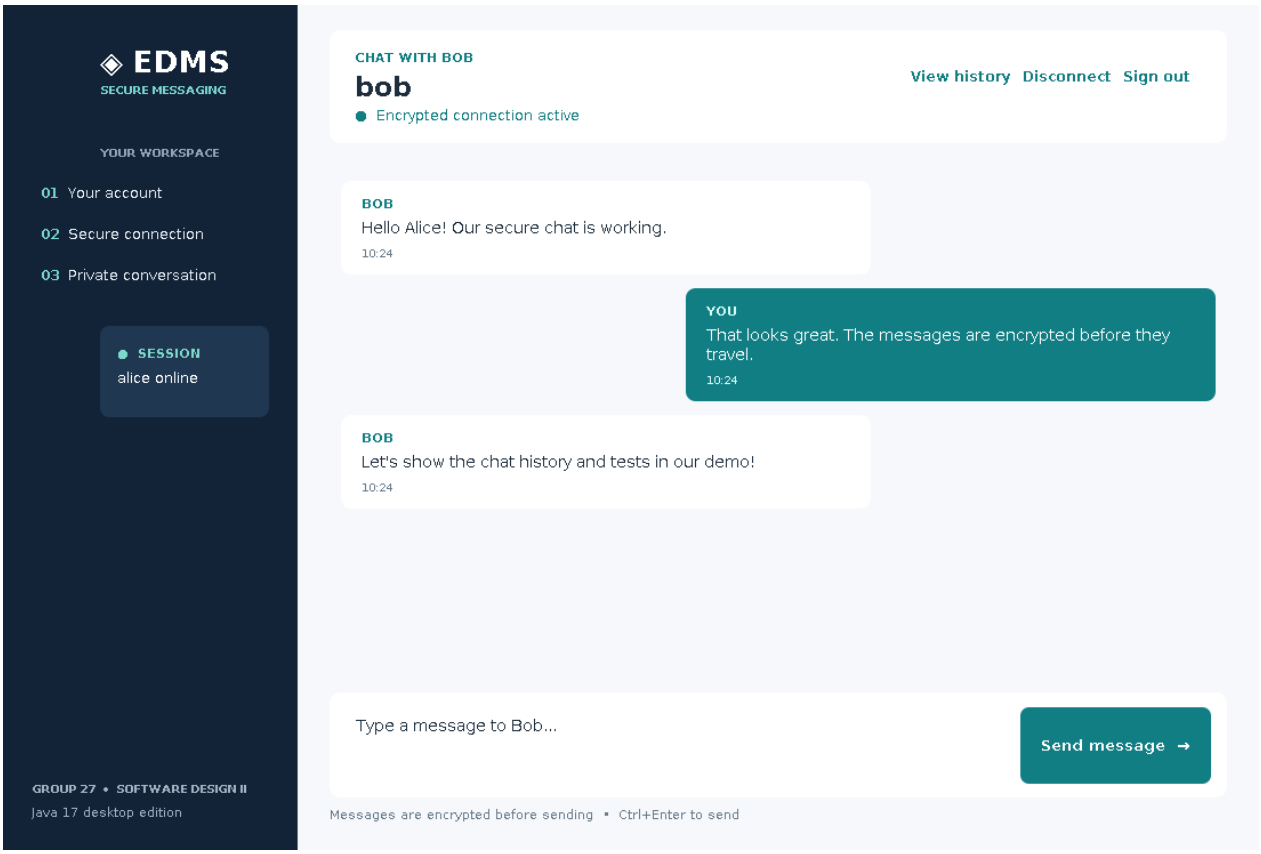


Figure 2. Swing desktop chat preview with sample conversation.

GUI implementation

EdmsGui uses SwingWorker and a dedicated receiver thread so network waits and encryption do not block the interface. It shares EdmsPrototype services and the same encrypted handshake with the console view. A complete interactive GUI rehearsal on the group's desktop is still required.

Part A. Software Design Report

Module: SDN260S - Software Design II **Project:** Encrypted Direct Messaging System (EDMS) **Group:** 27 **Version/date:** Java 17 Swing GUI trial, 26 October 2026 **Status:** Group review required before academic submission

1. Problem definition

Ordinary chat interfaces may expose sensitive information if messages travel or are stored without suitable protections. The group's problem was to design a small direct-messaging application that authenticates local users, exchanges information between two computers/processes through a network interface, encrypts messages, and keeps an accessible chat history. The project also exercises software design principles: object-oriented classes, modularity, file handling, networking, error handling and testing.

The original console prototype attempted these tasks but contained malformed Java source, fixed demonstration users, a hard-coded AES key, AES-ECB encryption and plaintext logging. It therefore required both functional repair and security improvement before a credible demonstration. After the console version passed its tests, the group requested a graphical desktop version modeled around a chat window.

2. Objectives and requirements

ID	Requirement	Implemented evidence
F01	Register and log in as a user	`AuthManager`; tests T01-T08
F02	Reject invalid/duplicate credentials	Input validation, constant-time hash comparison; T02-T06
F03	Send/receive direct messages	TCP `NetworkManager`; T19 and two-terminal run
F04	Protect message confidentiality/integrity	`EncryptionService` AES-GCM; T09-T13
F05	Store and retrieve history	`ChatHistory` + `FileStorage`; T17-T18
F06	Provide user commands and orderly exit	`/history`, `/help`, `/quit`, `/logout`; live run
F07	Log operational events without exposing message bodies	`AuditLogger`; event-only logging
F08	Test normal, boundary, malformed and disconnected states	T01-T20, manual interaction
F09	Operate through a graphical interface	`EdmsGui`: account screens, connection setup, chat bubbles, history viewer, logout; preview test T23
NF01	Readable modular OOP code	Named responsibility classes and updated UML
NF02	Run without third-party dependencies	Java standard library; build script
NF03	Limit malformed input and oversized messages	Message and socket frame bounds
NF04	Keep graphical interface responsive during I/O	Swing event dispatch thread, background workers and dedicated receive thread

Scope: A localhost, two-user, one-peer-per-session educational desktop application. The original console version remains available; the new Swing GUI offers screens for authentication, connection and chatting, with a history viewer and session controls.

3. Architecture and class responsibilities

The updated UML and sequence model are in UML_UPDATED.md. The `EdmsPrototype` entry point integrates the following:

- **User and AuthManager:** persistent account identity, unique usernames, individually salted PBKDF2-SHA256 password verification, and signed-in `Session`.
- **Message:** immutable message value with UUID, timestamp, sender, receiver and text. Serialization uses Base64 for content to avoid delimiter ambiguity.
- **EncryptionService:** derives a 256-bit AES key using PBKDF2-HMAC-SHA256 and provides authenticated AES-GCM encryption/decryption. A new random 12-byte IV is used for every encrypted record.
- **NetworkManager:** loopback server/client socket, bounded newline-delimited frame transport, connection timeout and close behaviour.
- **ChatHistory and FileStorage:** per-user encrypted files, decrypting only after a successful local login. Corrupted lines are counted and skipped while valid lines remain readable.
- **AuditLogger:** timestamps account, connection and message-handling events using Java logging; never writes message content, account passwords or chat passphrases.
- **Application driver:** displays menus, performs encrypted peer hello, checks sender/receiver values, starts a receiver thread and dispatches console commands.
- **Swing desktop GUI (`EdmsGui`):** presentation layer for account registration, login, server/client selection, encrypted chat, message history, disconnect and logout. It reuses the original services and the shared `secureHandshake` method instead of introducing a second message protocol.

Graphical interface design

The GUI uses the Java Swing toolkit with a sidebar and three main states: **Account to Connection to Conversation**. The account screen handles registration/login; the connection screen lets a user start a localhost listener or join as a client, and the conversation screen provides message bubbles, timestamps, a text composer, a history dialog, and session controls. The `SwingWorker` mechanism keeps password verification, connection establishment and message sending off the graphical event dispatch thread. Incoming TCP frames are read on a background daemon thread and safely displayed using `SwingUtilities.invokeLater`. A headlessly rendered layout image is included as `docs/GUI_PREVIEW.png`. This proves that the Swing chat layout renders, but not that all mouse interactions have been exercised on a desktop machine.

Data flow

- A user registers/logs in. A random password salt and hash live in `users.tsv`; the original password is not persisted.
- Alice starts a localhost server and Bob connects with a client. Both enter the same shared chat passphrase **out of band**.
- Alice sends a public 16-byte random session salt. Both derive matching transport keys using PBKDF2. They exchange AES-GCM encrypted `HELLO` messages before chat begins.

- Alice creates a `Message`, serializes it, encrypts its complete body and metadata, then sends an `ENC|` Base64 frame.
- Bob checks the GCM authentication tag, validates sender/receiver and reconstructs the message. The chat is displayed and a separately encrypted history entry is saved under Bob's account.
- Both can inspect local history with `/history` or end the session with `/quit`.

4. Technical choices and algorithms

Authentication: Passwords are checked using PBKDF2-HMAC-SHA256 (120,000 iterations) and random per-account 16-byte salts. Hashes are compared with `MessageDigest.isEqual`. Different salts produce different stored hashes even for identical passwords. Registration enforces a 3-20-character username policy and an 8-128-character password length.

Transport encryption: AES-256 in Galois/Counter Mode (`AES/GCM/NoPadding`) provides confidentiality plus a 128-bit authentication tag. Nonces are random per encryption operation. Corrupt payloads or wrong shared keys are rejected, unlike the prototype's unauthenticated ECB mode. The session salt is public; the passphrase must be kept private.

History encryption: A distinct key is derived from the signed-in user's account password and a separate persistent history salt. The application saves one Base64-encoded AES-GCM value per file line. This avoids plaintext chat logs. It also means lost passwords cannot recover old history in this prototype.

Storage: `users.tsv` is replaced as a whole using a temporary file and atomic move where supported. Message history is appended; invalid records are skipped on reading instead of stopping the session. Standard Java I/O handles resource cleanup.

Network: Java `ServerSocket` and `Socket` transport bounded lines over localhost. The initial socket read has a 10-second timeout and connection uses a 5-second timeout; no socket read timeout remains after the handshake so users may pause while typing. A dedicated receiver thread allows incoming and outgoing chat concurrently.

5. Error handling and usability

The app reports invalid input, duplicate accounts, failed login, weak passphrase, inaccessible port, mismatched key, corrupt history, malformed cipher and network disconnect. It accepts Unicode text, rejects empty/oversized messages and supports `/help`. Operational logs are kept separately from encrypted conversations. The console UI avoids dependencies and is practical for two-terminal demonstration, though asynchronous incoming text may visually interrupt typing.

6. Testing and evaluation

The automated runner covers **23 cases**, including normal registration/login, duplicate and bad inputs, persistence, salted hashes, Unicode round trips, random IVs, tampering, wrong key, message size boundaries, history corruption, socket disconnect, peer identity handshake, wrong shared passphrases and headless Swing layout rendering. The verified result in an environment permitting localhost sockets was **23 passed, 0 failed**. A separate live Alice/Bob console run verified registration, encrypted connection, two-way messages, history on both sides and exit without crashing. Detailed observed outputs are in TEST_EVIDENCE.md. A physical GUI click-through rehearsal remains a pre-submission task because the automated build environment has no desktop display.

The initial restricted test runner blocked loopback port creation with `Operation not permitted`; the tests were rerun in an authorized environment allowing localhost sockets. This was a sandbox restriction, not a code failure.

7. Changes from the group prototype

Original	Revised
Escaped/malformed Java source	Builds cleanly on Java 17
Built-in Alice/Bob accounts	Interactive registration and persistent accounts
Unsalted SHA-256 password digest	Random-salt PBKDF2 password hashing
Hardcoded AES key + ECB mode	Prompted shared passphrase, AES-GCM authentication
Plaintext `chat_history.log`	Per-user encrypted `.history` records
One-off crypto self-checks	Twenty-three repeatable automated tests
Unbounded incoming lines	16,384-character frame limit
No implementation-matched report	Updated UML, user guide, demo script and design report
NetBeans console only	Optional Swing desktop registration, connect and chat screens; console retained

8. Security, societal impact and limitations

The design demonstrates more responsible handling of confidential information, but it **must not be described as production-secure**. The application has no trusted remote identity registry, certificate validation, ratcheting/forward secrecy, independent key exchange, replay prevention, safe password recovery, rate limiting, encrypted backups, central server routing or support for many simultaneous peers. A weak shared passphrase may be guessed offline from captured authenticated ciphertext. The registration file is not designed for concurrent updates across machines. Only localhost connections are deliberately allowed.

Security and usability must be balanced: protecting history raises the risk of losing data when a password is forgotten; asking users to exchange a shared passphrase introduces a social-engineering risk; diagnostics should never collect private message bodies. Future development should use an established authenticated key agreement protocol, stronger account lifecycle controls, file permissions/locking, abuse limits, accessibility improvements and independent security review.

9. Team responsibilities - verify before submission

Names/roles here are taken from the group's supplied source comments, **not proof of who completed every revised line**. Each member should confirm and accurately describe their own contribution and the work done collaboratively.

Member	Responsibility listed in original prototype	Implemented area to explain
Aaron Majova	User accounts & authentication	`User`, `AuthManager`, login/register
Bonolo Mashiane	Console interface	Original console input plus GUI presentation to review with the team
Aphiwe Nokuphiwa Mchunu	Encryption	AES-GCM encryption path
Londeka Mthengwana	Decryption and key management	PBKDF2 keys, authenticated decryption
Amahle Mthethwa	Networking	TCP sockets, framed messages and disconnects
Nomusa Ndumo	File handling & message storage	`ChatHistory`, `FileStorage`, audit logs
Tshegofatso Seleko	Testing and security validation	`EdmsTests`, normal and edge tests
Angel Nokulunga Skhosana	Integration and driver	Main application, end-to-end behaviour

10. Conclusion and recommendations

The improved EDMS meets the identified **core console demonstration** requirements and adds a functioning compiled/rendered Swing front end: local account management, encrypted two-way messaging, protected history, validation, logging, a UML-linked modular design and repeatable test evidence. It is ready for **GUI rehearsal** on the group's actual presentation laptop; the full desktop interaction has not yet been manually exercised in this headless build environment. A deployed multi-user messaging product would require the security and scalability work identified above.

References

- Oracle, *Java SE 17 Cryptography Architecture*, `Cipher`, `GCMParameterSpec`, `SecretKeyFactory`, `ServerSocket` and `Socket` API documentation: https://docs.oracle.com/en/java/javase/17/security/java-cryptography-architecture-jca-reference-guide.html
- Engineering Council of South Africa (ECSA), *Qualification Standard for Diploma in Engineering (NQF Level 6), E-02-PN*, GA2 and GA7. https://www.ecsa.co.za/download/483/new-engineering-technology-qualification-standards/1007434/e-02-pn-qualification-standard-for-diploma-in-engineering-nqf-level-6-2.pdf
- South African Qualifications Authority, registered qualification description with graduate-attribute statements: https://regqs.saqa.org.za/viewQualification.php?id=118411

Part B. Test Evidence

Test date: 26 October 2026 **Environment:** OpenJDK 17.0.20, Linux, Java standard library **Command:** `bash scripts/test.sh` **Observed outcome: 23 PASS / 0 FAIL**, including real localhost socket/handshake tests and Swing layout rendering performed with permitted socket access.

1. Normal and edge-case test matrix

ID	Type	Input/action	Expected result	Observed
T01	Normal	Register `alice` / `password123`	Account created	PASS
T02	Edge	Register `alice` twice	Duplicate rejected	PASS
T03	Invalid	Register `A!`	Username rejected	PASS
T04	Boundary	Password `123`	Too-short password rejected	PASS
T05	Normal	Correct Alice login	Login permitted	PASS
T06	Invalid	Incorrect password and unknown username	Both rejected	PASS
T07	Persistence	Reopen account store then log in	Login still works	PASS
T08	Security	Same passwords for two accounts	No plaintext, different salted hashes	PASS
T09	Normal	AES-GCM with Unicode `[lock], [ZA]`	Decrypted text exactly matches	PASS
T10	Security	Encrypt the same text twice	Ciphertexts differ (random nonces)	PASS
T11	Invalid	`Not-base64!!` ciphertext	Checked rejection, no crash	PASS
T12	Security	Flip one ciphertext byte	Authentication failure	PASS
T13	Security	Decrypt with wrong passphrase	Authentication failure	PASS
T14	Normal	Message with pipes, tab, Unicode	Message metadata/content preserved	PASS
T15	Boundary	Blank, 2,001 and 2,000-character messages	First two rejected; last accepted	PASS
T16	Invalid	Incomplete fields and bad sender	Message rejected	PASS
T17	Normal	Save and reload private history	Message recovered; plaintext absent from file	PASS
T18	Recovery	Append corrupted history and try wrong key	Good entry readable; invalid entries skipped	PASS
T19	Integration	Loopback `PING`/`PONG`, then disconnect	Both directions succeed; EOF detected	PASS
T20	Invalid	Port `70000`, incomplete client args	Helpful validation failures	PASS
T21	Integration	Two-peer AES-GCM hello and encrypted message	Both peer identities and encrypted message verified	PASS
T22	Security	Two peers use different shared passphrases	Both authenticated handshakes rejected	PASS
T23	GUI	Render Swing `ChatPanel` while headless	1100×740 screenshot with visible sidebar and chat panels	PASS

The test program uses assertions and exits with non-zero status if any case fails; a printed `PASS` is not a manually assigned label.

2. Full automated test output (observed)

```
Build successful (Java 17).
PASS T01 | normal registration succeeds
PASS T02 | duplicate registration safely rejected
PASS T03 | invalid username rejected
PASS T04 | short password rejected
PASS T05 | correct login succeeds
PASS T06 | incorrect and absent logins rejected
PASS T07 | user accounts survive restart
PASS T08 | credential storage has salt, not plaintext
PASS T09 | AES-GCM round trip incl. Unicode
PASS T10 | random nonce changes ciphertext
PASS T11 | malformed ciphertext rejected
PASS T12 | tampered authenticated ciphertext rejected
PASS T13 | wrong shared passphrase rejected
PASS T14 | message serialization survives punctuation and Unicode
PASS T15 | blank/oversize messages rejected; exact limit accepted
PASS T16 | malformed message fields rejected
PASS T17 | encrypted history save and reload
PASS T18 | wrong history key and corrupt entry handled
PASS T19 | network loopback transfer and disconnect
PASS T20 | invalid CLI ports and incomplete args rejected
PASS T21 | encrypted two-peer handshake and chat frame
PASS T22 | mismatched peer passphrases abort handshake
PASS T23 | Swing chat view renders a GUI preview headlessly
---------------------------------------
RESULT: 23 passed, 0 failed (23 cases)
```

During the first (earlier console-version) run in a restricted execution sandbox, T19 could not bind to localhost (`Operation not permitted`). The tests were rerun in a permitted localhost environment and passed. With the new GUI/handshake tests added, the latest authorized run returned **23/23**. A standard desktop environment is needed to click through the GUI itself; T23 verifies actual Swing layout rendering without a display.

3. Live two-terminal functional demo (observed)

Actual integration run used **Alice as server** and **Bob as client** with port `8891`, a throwaway `demo-data` directory, and the same manually entered shared passphrase. Both users were registered through the app UI in their respective windows.

Terminal A (Alice/server):

```
Registration successful.
Signed in as alice.
Waiting for peer on localhost:8891 ...
Encrypted connection ready: alice <-> bob
Type a message, /history, /help or /quit.
> Hello Bob, this is Alice [lock]
[Sent securely to bob]
>
[2026-10-26T13:16:04.507736359Z] bob: Hi Alice, encryption works [check]
> /history
--- Encrypted local chat history (decrypted for signed-in user) ---
[2026-10-26T13:15:53.030735683Z] SENT alice -> bob: Hello Bob, this is Alice [lock]
[2026-10-26T13:16:04.507736359Z] RECEIVED bob -> alice: Hi Alice, encryption works [check]
> /quit
Logged out. EDMS shutdown complete.
```

Terminal B (Bob/client):

```
Registration successful.
Signed in as bob.
Connecting to localhost:8891 ...
Encrypted connection ready: bob <-> alice
Type a message, /history, /help or /quit.
>
[2026-10-26T13:15:53.030735683Z] alice: Hello Bob, this is Alice [lock]
> Hi Alice, encryption works [check]
[Sent securely to alice]
> /history
--- Encrypted local chat history (decrypted for signed-in user) ---
[2026-10-26T13:15:53.030735683Z] RECEIVED alice -> bob: Hello Bob, this is Alice [lock]
[2026-10-26T13:16:04.507736359Z] SENT bob -> alice: Hi Alice, encryption works [check]
> /quit
Logged out. EDMS shutdown complete.
```

The original console stream contained extra prompt redraws when incoming and typed text overlapped; the transcript above omits those prompt-only artifacts. No message content was changed.

4. Evidence-to-requirement summary

Requirement	Demonstration/test evidence
Account management	Registration in both terminals; T01-T08
Two-way messaging	Alice to Bob and Bob to Alice live transcripts; T19
Encryption and authentication tags	T09-T13 and full encrypted handshake T21-T22; AES-GCM implementation
Encrypted local history	`/history` on both terminals; T17, T18
Usability and errors	T02-T04, T15-T16, T20, `/help` and `/quit`
Graceful socket close	Both live sessions exited normally; T19
Desktop GUI layout	T23 image rendering and `docs/GUI_PREVIEW.png`

5. New Swing GUI preview evidence

This preview was produced by `java -Djava.awt.headless=true -cp out EdmsGui --preview docs/GUI_PREVIEW.png` using the same `ChatPanel` component as the application. It is **not** evidence of clicking through a live desktop session. Registration, connection controls and chat interactions must be rehearsed on the group's Windows/NetBeans laptop before submission.

6. Known untested or future scenarios

These results do **not** prove independent remote identity, resistance to replay or distributed denial-of-service, cross-machine operation, persistence after simultaneous account updates by separate processes, many-to-many messaging, or production security. These are listed as future work in the software design report. A lecturer may ask for screenshots on the group's own laptop; capture those during the final rehearsal using the supplied demo script.

Part C. GA2 and GA7 Reflection

Important: This is a **starting draft** written around the software design and verified tests, not a claim that every member personally did every task. Adjust the first-person statements to reflect the group's actual activities, disagreements, challenges, individual contributions and university rubric before submitting.

GA2 - Application of scientific and engineering knowledge

For our Encrypted Direct Messaging System, we applied computer science and software engineering concepts to a practical communication problem. We decomposed the application into account management, messages, encryption, networking, history storage and integration. The UML class design helped us identify what information and behaviour each component should own. This separation made the program easier to understand and test: for example, `ChatHistory` manages the user's view of saved conversations while `FileStorage` handles the actual file operations.

The strongest example of applied knowledge was encryption. Our first prototype used AES in ECB mode with a fixed key. In reviewing it, we recognized that making a program "encrypt text" is different from providing meaningful protection. We replaced that mechanism with AES-GCM, a mode that combines encryption and integrity verification. We also used random nonces so encrypting the same input twice produces different ciphertexts. For account authentication, we applied PBKDF2-HMAC-SHA256 with randomly generated salts instead of directly hashing passwords with SHA-256. These choices required knowledge of cryptography, randomness, secure key derivation, byte encodings and Java's cryptographic APIs.

We also applied networking knowledge through TCP socket programming. We considered how a server waits for a peer, how both ends exchange encrypted frames, and what happens when a connection closes unexpectedly. We used basic data-structure and algorithm design skills to give messages unique identifiers and timestamps, serialize the fields safely, and reject malformed or oversized inputs. Separating a receiving thread from the sending interface demonstrates understanding of concurrent operations.

Our testing approach connected theory to evidence. We did not only test a typical successful message: we checked duplicate account creation, wrong credentials, Unicode text, an exactly 2,000-character message, an oversized message, altered ciphertext, the wrong key, corrupt history and socket disconnection. The final automated result was **20/20 passing tests** under permitted localhost execution, followed by a successful two-terminal Alice/Bob demonstration. The first socket test failed solely because the restricted execution environment prevented local port binding; rerunning under an environment that allowed sockets helped us distinguish an environmental restriction from an implementation defect.

We extended the console application with a Java Swing interface and learned to separate presentation work from the secure messaging services. Graphical controls initiate registration, background socket connections and message sending, while long-running operations are moved away from the event dispatch thread so the interface remains responsive. We generated a preview of the actual chat layout and added a rendering test, although real mouse/keyboard usability still needs to be checked on our laboratory computers.

One learning point was that modular code and an attractive demo alone are insufficient: algorithms, boundaries, error handling and repeatable verification must support the design. For further

improvement, we would research authenticated key exchange, formal threat modelling, synchronized file writes and usability testing, and ask others to review our cryptographic assumptions.

GA7 - Sustainability and impact of engineering activity

Our project deals with private communication, so its engineering decisions may affect personal safety, trust and dignity. A chat system that accidentally exposes passwords or sensitive messages could harm students, families or organizations. We therefore considered the impact of software not only on whether it functions, but on how it treats personal data. In the improved design, account passwords are represented by salted password-derived hashes, message content is encrypted over the network, and saved history is encrypted rather than written into plain diagnostic logs.

We recognized that security features introduce responsibilities and trade-offs. Strong encryption can protect a vulnerable user against casual interception, but a system using a shared passphrase may still be unsafe if the passphrase is weak or disclosed. Encrypting local chat records protects privacy, yet the absence of password recovery also risks permanent data loss. A user could falsely trust a peer identity because the application does not independently verify who knows the shared passphrase. It would be unethical to market this academic prototype as a production-secure messenger without confronting those limitations.

Accessibility and practical usability also matter. The original console mode runs on ordinary machines without commercial software or network services, which lowers cost and infrastructure needs for a teaching demonstration. However, it may not suit users unfamiliar with command lines. The new Swing version introduces visible forms, connection-mode choices, readable conversation bubbles and buttons for history, disconnect and sign out. A future version should test keyboard-only accessibility, screen-reader announcements, high-contrast modes and localization, along with better identity checking, secure backups and abuse controls.

There are economic and environmental implications. A small Java-standard-library prototype needs no paid hosting, cloud database or third-party messaging service for a local demonstration, reducing costs and resource use. That does not mean it has zero environmental impact: development computers consume electricity, and large-scale deployment would require evaluating energy use, storage retention and the cost of maintaining encryption and security updates.

The key GA7 lesson is to consider the people affected by design choices. Protecting confidentiality, limiting logs, documenting risks and testing failure cases are part of responsible engineering. If we extended EDMS beyond an academic demonstration, we would consult users, study privacy and accessibility requirements, review relevant laws and standards, and arrange an independent security assessment before allowing real sensitive communications.